

05 — Memory Forensics

Injection clues, process lists, and malfind workflows

Educational / Defensive DFIR Study Notes

LEARNING OBJECTIVES

- Interpret RWX / private / MZ clues as injection leads
- Use pslist vs psscan (and friends) to find hidden or exited processes
- Apply malfind / vadinfo / dlllist / netscan in a logical order
- Know when and how analysts dump suspicious regions for file analysis

SCOPE REMINDER

- This material covers detection, artifacts, and incident response only.
- Attacker actions are phrased as 'adversaries may...' then pivot to evidence.
- Tool/command names appear as IOC strings / exam knowledge — not how-to guides.

Why memory matters

Fileless and injected payloads may never exist as a clean PE on disk. Credentials, beacon configuration, and process hollowing evidence often live primarily in RAM. Acquire memory early; analyze with Volatility 2/3, MemProcFS, or commercial EDR memory modules.

Injection / hollowing clues

- **RWX** — PAGE_EXECUTE_READWRITE is unusual for ordinary modules; often flagged by malfind
- **Private memory** — not mapped to a file on disk (contrast with normal DLLs)
- **MZ header** — PE in an unexpected memory region suggests reflective load or unpacking
- **Process hollowing** — adversaries may start a legitimate process then replace its memory; name looks trusted, memory map does not

Core plugins / commands (conceptual)

Plugin / view	Purpose	DFIR tip
pslist	Walk active process list structures	What the OS 'thinks' is running
psscan	Pool-tag / scan for EPROCESS objects	Reveals hidden, exited, or DKOM-unlinked processes — compare to pslist
pstree	Parent/child relationships	Spot odd parents (e.g., office → cmd → powershell)
cmdline	Process command lines from memory	Catch encoded PowerShell / dump-tool IOC strings
dlllist	Loaded modules	Missing on-disk path; unsigned; odd Temp paths
vadinfo / vadwalk	Virtual address descriptors	Permissions and mapped files per region
malfind	Heuristic: executable private VADs, often MZ	PAGE_EXECUTE_READWRITE + private + MZ = investigate
netscan / netstat	Network endpoints in memory	C2 IPs even if disk logs rotated

pslist vs psscan — exam favorite

If a process appears in **psscan** but not **pslist**, suspect hiding (e.g., DKOM) or a recently exited process still lingering in memory. Always reconcile both lists and investigate discrepancies.

malfind workflow

- Run malfind; prioritize regions with EXECUTE_READWRITE, not file-backed, containing MZ
- Dump the region (Volatility malfind dump options / windows.malfind --dump style workflows) to a file
- Hash the dump; run strings/FLOSS/VT/YARA; rebuild PE if needed
- Tie owning process to netscan and pstree for C2 and patient-zero path

How do I prove injection?

- Memory: malfind hit with RWX + MZ in process that should not have private PE images
- Correlate: EDR Sysmon 8/10 (CreateRemoteThread / ProcessAccess) if available around the same time
- Disk optional: dropper may be gone — memory may be the only proof
- Network: netscan shows unexpected remote endpoints owned by the hollowed/injected process

Common exam traps

- Treating every RWX region as malware (some legitimate software uses RWX briefly)
- Using only pslist and missing hidden processes
- Rebooting before memory acquisition when injection is suspected

- Ignoring cmdline in memory when disk Prefetch was cleared
- Forgetting to dump malfind regions for offline PE/YARA analysis

QUICK REFERENCE

- Clues: RWX + private + MZ
- pslist vs psscan → find hidden/exited/DKOM
- malfind → dump → strings/YARA/VT
- Also: pstree, cmdline, dlllist, netscan, vadinfo
- Memory first when fileless suspected

Mini Self-Check

Answer these without looking back. Answers are on the next page.

- Q1.** What three memory characteristics often appear together in malfind hits?
- Q2.** What does a process in psscan but not pslist suggest?
- Q3.** Why is process hollowing hard to spot by process name alone?
- Q4.** Which plugin helps link an injected process to C2 IPs still in RAM?
- Q5.** After malfind flags a region, what is the usual next forensic step?

Self-Check Answers

- A1. Executable permissions such as PAGE_EXECUTE_READWRITE (RWX), private (not file-mapped) memory, and an MZ/PE header.*
- A2. Possible process hiding (e.g., DKOM) or an exited process still recoverable — investigate the discrepancy.*
- A3. The process name/path may be a legitimate binary while the memory contents were replaced with malicious code.*
- A4. netscan (or equivalent network connection enumeration from memory).*
- A5. Dump the memory region to disk and analyze the dump (strings, YARA, rebuild PE, VT) while correlating process tree and network.*